

PROGRAMMIERUNG I

Kapitel 4: Fortgeschrittene Themen

Packages Und Visibility

Die Aufteilung der **Ducks** auf verschiedene Gehege hat zwar eine Zeit lang geholfen, doch schon im darauf folgenden Jahr sind die Kapazitäten des Zoos wieder voll ausgeschöpft.

Ein weiteres Gehege kommt nicht in Frage. Da die **Ducks** aber sicher auch in freier Wildbahn gut zurecht kommen, erwägt der Zoo die Implementierung der Methode `Zoo.freeAnimal()` zur Freilassung von Tieren:

```
public class Zoo {
    private Animal[] allAnimals = new Animal[30];

    public void freeAnimal(Animal animal) {
        System.out.println("Freeing Animal: " + animal);
        for (int i = 0; i < allAnimals.length; i++) {
            if (allAnimals[i].equals(animal)) {
                allAnimals[i] = null;
            }
        }
    }
}
```

Es gibt aber gerechtfertigte Bedenken, dass dann jedes Objekt (z.B. ein `ZooVisitor` oder `GreenpeaceActivist`) nach Belieben jede Art von Tieren befreien könnte.

Möglicherweise würden Tiere sogar Selbstbefreiungsversuche wagen:

```
public class Duck extends Bird
    implements Flying, Swimming, Omnivore, CarnivoreEatable {

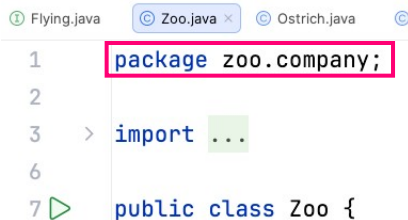
    public void freeMyself() {
        new Zoo().freeAnimal(this);
    }
}
```

Der Zoo muss also erstmal ordentlich abgesichert werden. Hier kommen **Packages** und **Visibility** ins Spiel.

Packages

Java-Programme können schnell eine Größe von mehreren hundert Klassen erreichen. Um bei einer derartigen Anzahl an Klassen/Interfaces die Ordnung zu wahren, können Java-Klassen und Interfaces in Packages geordnet werden. Die Angabe eines Packages ist zwar nicht zwingend, aber für jedes ernsthafte Projekt dringend zu empfehlen.

Das Package wird ganz am Anfang jeder Klasse angegeben.



```
1 package zoo.company;
2
3 import ...
4
6
7 public class Zoo {
```

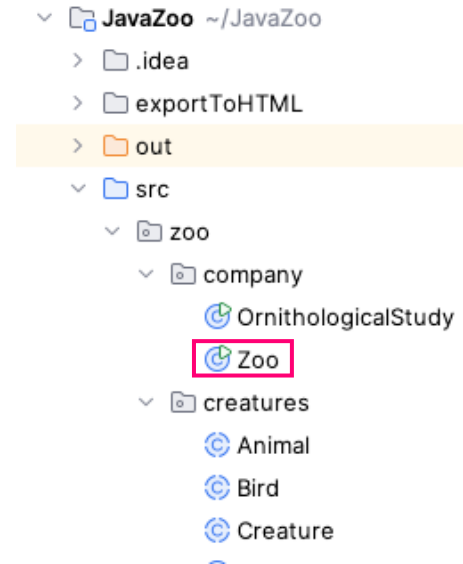
Packages sind hierarchisch strukturiert, d.h. ein Package kann weitere Sub-Packages enthalten. Bei der Definition werden die Hierarchieebenen mittels Punkten getrennt (*siehe oben*). Ein Package enthält üblicherweise mehrere Klassen/Interfaces und/oder Sub-Packages (*siehe rechts*).

Die *.java und *.class Dateien müssen zwingend in Unterordnern abgelegt werden, welche exakt der Package-Hierarchie entsprechen, z.B.:

JavaZoo/src/zoo/company/Zoo.java

Über die Angabe des voll-qualifizierten Klassennamens ist es möglich, verschieden Klassen zu unterscheiden, welche den gleichen Namen haben, z.B.: **java.util.List** und **java.awt.List**

Es ist Konvention den Package-Namen mit dem Domain-Namen zu beginnen, z.B.: **com.google** oder **org.apache**.



Visibility

Der Zugriff auf Klassen, Variablen, Methoden und Konstruktoren kann in Java durch vier **Visibility-Stufen** kontrolliert werden:

Visibility	Keyword	Erlaubte Zugriffe	Anwendungsbereich
private	<code>private</code>	1. Zugriff innerhalb der Klasse selbst	▪ Variablen ▪ Methoden/Konstruktoren
package (package-private ; default)	(kein Keyword)	1. Zugriff innerhalb der Klasse selbst 2. Zugriff von Klassen desselben Packages	▪ Variablen ▪ Methoden/Konstruktoren ▪ Klassen/Interfaces
protected	<code>protected</code>	1. Zugriff innerhalb der Klasse selbst 2. Zugriff von Klassen desselben Packages 3. Zugriff von Subklassen (egal in welchem Package)	▪ Variablen ▪ Methoden/Konstruktoren
public	<code>public</code>	1. Zugriff innerhalb der Klasse selbst 2. Zugriff von Klassen desselben Packages 3. Zugriff von Subklassen (egal in welchem Package) 4. Zugriff von jeder beliebigen Klasse (World)	▪ Variablen ▪ Methoden/Konstruktoren ▪ Klassen/Interfaces

Wenn Klassen auch von anderen Entwicklern verwendet werden sollen, kann durch Einschränkung der **Visibility** das Risiko für Fehlverwendung reduziert werden.

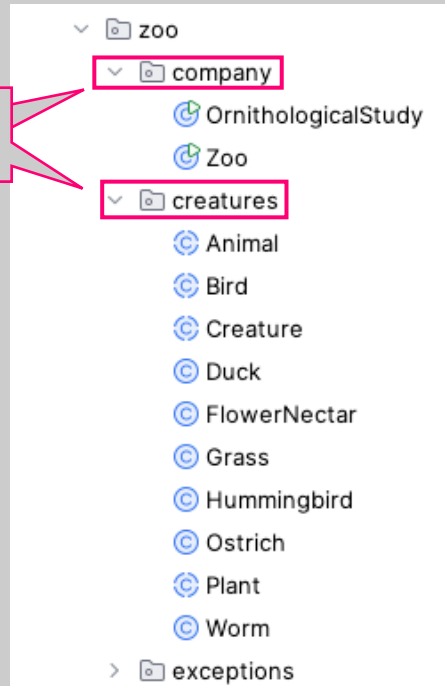
Generell ist es empfehlenswert, die restriktivste Visibility zu wählen, die für das gewählte Element Sinn ergibt. Für Instanz-/Klassenvariablen ist dies häufig `private`.

Anwendung von Packages und Visibility

Um den Zoo ordentlich abzusichern, sind zwei Schritte erforderlich:

Strukturierung der Klassen in Packages

Unterschiedliche
Packages



Einschränkung der Visibility von Methoden:

```
public class Zoo {
    private Animal[] allAnimals = new Animal[30];

    private void freeAnimal(Animal animal) {
        System.out.println("Freeing Animal: " + animal);
        for (int i = 0; i < allAnimals.length; i++) {
            if (allAnimals[i].equals(animal)) {
                allAnimals[i] = null;
            }
        }
    }
}
```

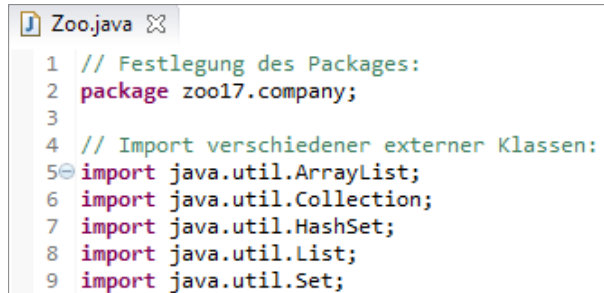
```
public class Duck extends Bird
    implements Flying, Swimming, Omnivore, CarnivoreEatable {

    public void freeMyself() {
        new Zoo().freeAnimal(this);
    }
}
```

The method freeAnimal(Animal) from the type Zoo is not visible

Importe

Wenn innerhalb einer Java-Klasse (z.B. **Zoo**) auf andere Java-Klassen oder Interfaces referenziert wird, welche sich in einem anderen Package befinden (z.B. **List**) dann müssen die referenzierten Klassen mittels des Keywords **import** eingebunden werden:

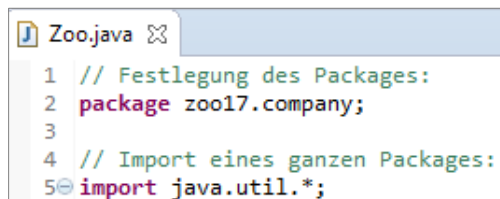


```
1 // Festlegung des Packages:
2 package zoo17.company;
3
4 // Import verschiedener externer Klassen:
5 import java.util.ArrayList;
6 import java.util.Collection;
7 import java.util.HashSet;
8 import java.util.List;
9 import java.util.Set;
```

Dadurch können diese Klassen im Code mit ihrem einfachen Namen (z.B. **List**) angesprochen werden.

Es ist möglich aber nicht allzu üblich, im Code die **voll-qualifizierten Klassen-Namen** zu verwenden (z.B. **java.util.List**) um sich entweder die Imports zu sparen oder verschiedene Klassen mit demselben Klassennamen zu unterscheiden (z.B. **java.awt.List**).

Es ist möglich aber konventionell unüblich, statt einzelner Klassen komplette Packages zu importieren, z.B:



```
1 // Festlegung des Packages:
2 package zoo17.company;
3
4 // Import eines ganzen Packages:
5 import java.util.*;
```

JavaDoc

Um anderen Entwicklenden das Verstehen des Quellcodes zu erleichtern, sollte man den geschriebenen Code nicht nur mit Inline-Kommentaren versehen, sondern (*viel wichtiger noch*) mittels **JavaDoc** dokumentieren:

Dokumentation
von **Klassen**

Dokumentation
von **Variablen**

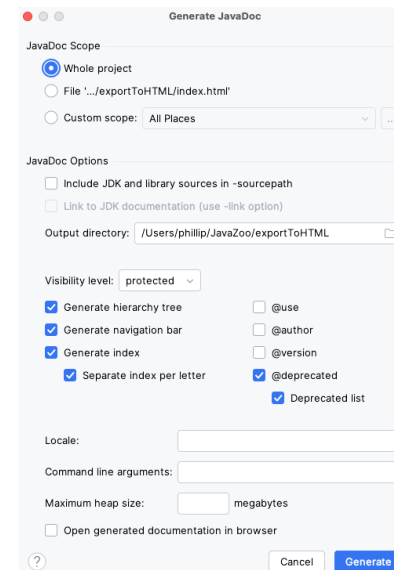
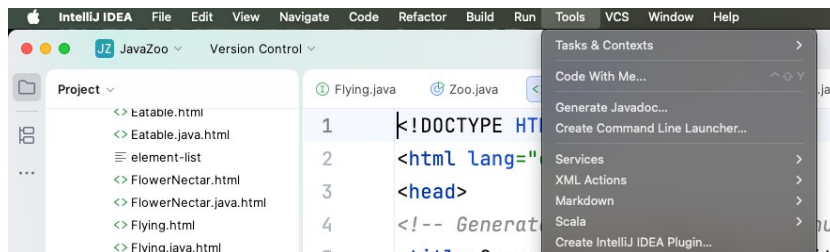
Dokumentation
von **Methoden**

```
/**
 * A Zoo, which contains an Array of {@link Animal}s.
 * @author Phillip Goellner
 * @see Animal
 */
public class Zoo {

    /** The array, which contains all {@link Animal}s of the {@link Zoo}. */
    private Animal[] allAnimals = new Animal[30];

    /**
     * Removes the specified {@link Animal} from the {@link #allAnimals} array.
     * @param animal The animal to be freed.
     */
    private void freeAnimal(Animal animal) {
        System.out.println("Freeing Animal: " + animal);
        for (int i = 0; i < allAnimals.length; i++) {
            if (allAnimals[i].equals(animal)) {
                allAnimals[i] = null;
            }
        }
    }
}
```

Beim Überfahren von Klassen, Variablen und Methoden mit der Maus zeigt IntelliJ automatisch das entsprechende JavaDoc an. Darüber hinaus kann IntelliJ auch alle JavaDocs eines Projekts in **HTML-Dokumente** exportieren:



Debugging

Während der Entwicklung kommt es häufig vor, dass nicht offensichtliche Fehler auftreten. Um diese beheben zu können, ist es sehr nützlich langsam durch die Ausführung vom Programm zu schreiten und zu analysieren was wann wo passiert.

Der Prozess der Fehlersuche und -behebung wird üblicherweise als **Debugging** bezeichnet. Ein nützliches Werkzeug dafür ist ein sog. **Debugger**, welchen man in jeder üblichen Entwicklungsumgebung vorfindet. Üblicherweise liefert ein Debugger folgende Informationen:

- Wo man sich gerade im Programm befindet
- Welche Werte gerade in den Variablen gespeichert sind
- Methodenaufrufe bisher

Um in das Programm so analysieren zu können, muss als erstes mindestens ein **Breakpoint** (die Stelle, an der die Ausführung angehalten werden soll) in das Programm eingefügt werden, und dann die Ausführung im Debug-Modus gestartet werden. Sobald sich das Programm am ersten Breakpoint befindet, kann anschließend schrittweise fortgefahren werden oder bis zum nächsten Breakpoint.

The screenshot shows an IDE window with a Java file named `Zoo.java`. The code defines a `Zoo` class with two methods: `breedingSeason()` and `listAllChildren()`. A breakpoint is set on line 126, which is the start of an `if` statement inside `listAllChildren()`. The IDE is in debug mode, and the execution has stopped at the breakpoint. A call stack on the left shows the current frame is `listAllChildren:126, Zoo`. The debug console on the right shows the current state of variables: `this` is `(Zoo@831)`, `animal` is `(Duck@846) "Hi (quak) my name is Freddy and I am 0 years old"`, and `allAnimals` is `(Animal[30]@832) ... Explore Elements`. Several callouts point to specific features: 'Ausführen im Debug-Modus' points to the debug button in the top toolbar; 'Die Ausführung befindet sich gerade hier' points to the breakpoint on line 126; 'Ausführungs-Kontrolle' points to the debug toolbar; 'Methoden-Aufrufe' points to the call stack; and 'Lokale Variablen' points to the variable view in the debug console.

Ausführen im Debug-Modus

Die Ausführung befindet sich gerade hier

Ausführungs-Kontrolle

Methoden-Aufrufe

Lokale Variablen

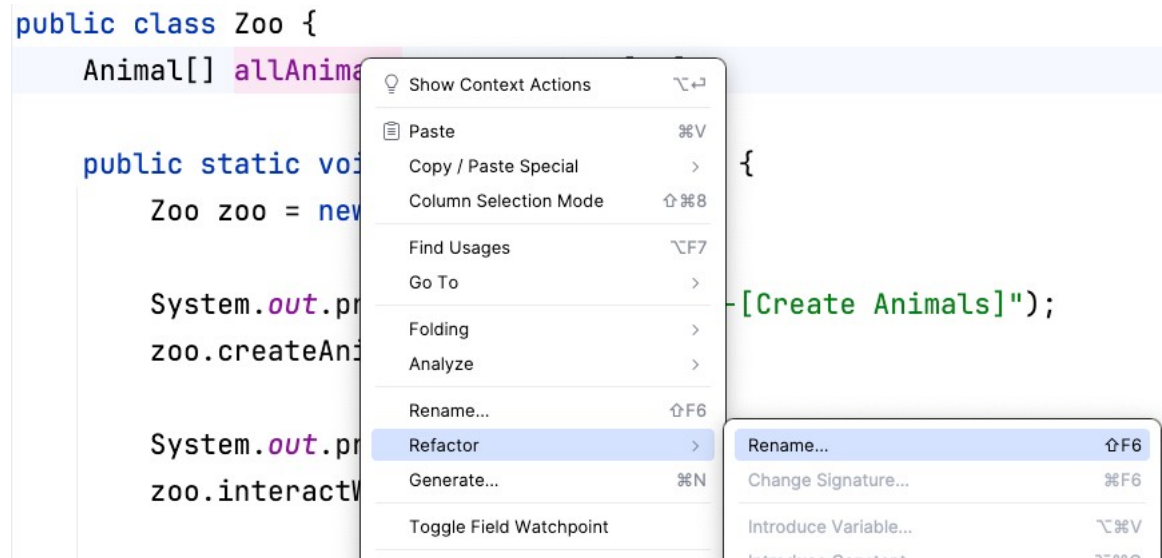
Refactoring

Häufig fällt während des Programmierens auf, dass z.B.:

- eine existierende Variable oder Methode umbenannt werden sollte.
- eine existierende Klasse umbenannt oder in ein anderes Package verschoben werden sollte.
- eine existierende Variable oder Methode in eine andere Klasse (z.B. Superklasse) verschoben werden sollte.

Derartige Anpassungen, bei welchen sich zwar die Struktur des Quellcodes ändert, aber nicht dessen Funktionalität, heißen **Refactoring**. Bei der Entwicklung von Libraries/Frameworks wird meist besonderer Wert darauf gelegt, dass beim Refactoring die Rückwärtskompatibilität der public APIs erhalten bleibt. Beim Refactoring muss häufig eine große Menge an Code geändert werden.

Zur Vereinfachung bieten IDEs wie Eclipse umfangreiche Werkzeuge an, mittels welcher das Refactoring größtenteils automatisiert werden kann (z.B. das Umbenennen von `allAnimals`):



Software-Testing

Um Software von hoher Qualität erstellen zu können, ist es essentiell sicherzustellen, dass sie sich nicht auf unerwartete Weise verhält (aka. Bugs enthält).

Die zentrale Tätigkeit, die das sicherstellt ist **Software-Testing**, die sich wiederum in verschiedene Praktiken aufteilt:

Statische Maßnahmen Der Quellcode wird manuell (z.B. im Peer Review) oder automatisiert (z.B. mittels Tools wie Sonar) überprüft, ohne dass die Software tatsächlich ausgeführt wird.

Dynamisches Testing Die Software wird tatsächlich ausgeführt und das tatsächliche Verhalten während der Testfälle wird mit dem erwarteten Verhalten verglichen.

Die verschiedenen Arten von Dynamischen Tests können anhand verschiedener Dimensionen klassifiziert werden (*nicht Teil dieser Vorlesung*). Zu den meist verwendeten Test-Begriffen gehören:

- **Unit/Module/Component Test** Überprüfung einzelner Methoden, Klassen oder Module/Packages.
- **Integration Test** Überprüfung kompletter Abläufe über mehrere Klassen/Module hinweg.
- **System Test/System Integration Test** Überprüfung des kompletten Systems auf die korrekte Umsetzung der fachlichen Anforderungen und der erwarteten Qualitätsmerkmale – meist durch die Entwickler, ggf. unterstützt durch GUI Testing Tools.
- **User Acceptance Test/Abnahmetest** Überprüfung des kompletten Systems durch den Auftraggeber als Voraussetzung für die Abnahme und damit die Bezahlung der Software.
- **Regression Test** Überprüfung des Systems nach einer Änderung/Wartungsmaßnahme auf neu hinzugekommene Fehler in Bereichen, die bisher funktioniert haben.
- **Penetration Test** Überprüfung des Systems auf Sicherheitsschwachstellen.

Assertions

Die kleinste Einheit im Testing ist die Überprüfung, ob gewisse Konditionen zutreffen oder nicht. Dafür werden sogenannte **Assertions** verwendet (aus dem Englischen `assert` $\hat{=}$ zusichern). Assertions treffen Aussagen über Code, die alle zutreffen müssen, damit das Programm ordnungsgemäß funktioniert.

Viele Programmiersprachen bieten eine native Möglichkeit Assertions zu implementieren. In Java wird dies getan, indem mit dem Keyword `assert` Konditionen überprüft werden. Falls die Kondition `false` ist, stürzt das Programm mit einem Fehler ab und der spezifizierte Text wird angezeigt:

Syntax:

```
assert Condition: "Text im Fehlerfall";
```

Obwohl Java über `assert` Statements verfügt, können sie leider nicht ohne Weiteres verwendet werden. Damit sie funktionieren, muss dem JDK die Option „enableassertions“ übergeben werden (`-ea`).

```
Exception in thread "main" java.lang.AssertionError: was not "Fred"
    at Duck.main(Duck.java:35)
```

Fehlermeldung bei einer fehlgeschlagenen Assertion

Unit-Tests

Die kleinsten logischen Bausteine in einem Programm – häufig einfache Methoden – werden über Unit-Tests getestet. Dabei ist das Ziel sicherzustellen, dass die spezifische Teil-Funktionalität dieser Methode wie erwartet funktioniert.



Gute Unit-Tests folgen dem FIRST-Prinzip:

- **Fast**
Das Ausführen eines Unit-Tests ist schnell und dauert nur wenige Millisekunden
- **Independent**
Unit-Tests sind nicht von externen Bedingungen abhängig
- **Repeatable**
Unit-Tests haben mit den selben Vorbedingungen immer das gleiche Ergebnis
- **Self-Validating**
Das Ergebnis eines Unit-Tests zeigt an, ob der Code funktioniert oder nicht; das Test-Ergebnis muss nicht weiter validiert werden
- **Timely**
Unit-Tests werden immer vor dem zu testenden Code geschrieben

```
void addition_calculates_correct_result() {  
    int a = 1;  
    int b = 3;  
  
    int additionResult = addition(a, b);  
  
    assert additionResult == 4: additionResult + " was not equal to expected result 4";  
}  
  
void addition_is_commutative() {  
    int a = 3;  
    int b = 1;  
  
    int additionResult = addition(a, b);  
  
    assert additionResult == 4: additionResult + " was not commutative";  
}
```

Beispiele für Unit-Tests

Test-Struktur

Eine nützliche Struktur, an der man sich beim Verfassen von Tests orientieren kann, bietet **Given-When-Then** (auch bekannt als [Gherkin-Syntax](#)). Mit dieser formalisierten Vorlage wird sichergestellt, dass die daraus resultierenden Tests leicht lesbar und eindeutig nachvollziehbar sind.

Given definiert alle **Vorbedingungen** (z.B. Test-Eingabewerte) und stellt so den Ausgangszustand vor dem eigentlichen Test her.

When ruft das zu testende Verhalten auf. Üblicherweise wird hier das Ergebnis (**Actual**) gespeichert, um anschließend überprüft zu werden.

Then definiert alle **Nachbedingungen**. Das ist üblicherweise das Überprüfen aller errechneten Ergebnisse.

```
void concatenating_creates_concatenated_string() {  
    // Given  
    String leftPart = "Hello ";  
    String rightPart = "World";  
  
    //When  
    String actual = concatenateStrings(leftPart, rightPart);  
  
    // Then  
    assert actual.equals("Hello World"): actual + " was not equal to \"Hello World\"";  
}
```

Akzeptanztests

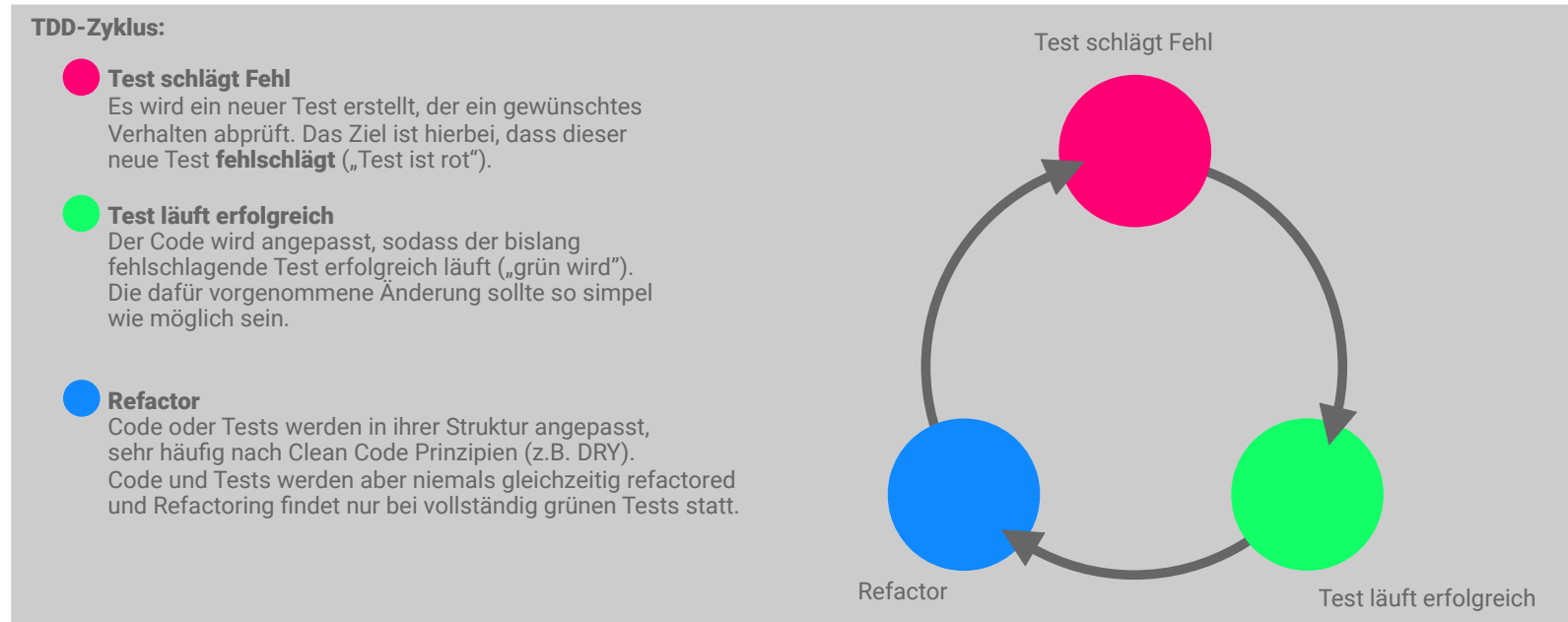
Im **Requirements-Engineering** gibt es das Konzept von **Akzeptanz-Kriterien**, welche die gewünschten Eigenschaften eines Produktes (Feature oder auch ganze Programme) beschreiben. Wenn eine Software alle Akzeptanz-Kriterien erfüllt, kann sie aus einer funktionalen Sicht als abgeschlossen betrachtet werden.

Akzeptanz-Kriterien werden während der Entwicklung als **Akzeptanztests** implementiert. Anders als Unit-Tests, prüfen Akzeptanztests häufig das Zusammenwirken verschiedener Funktionalitäten und operieren daher auf einer höheren Abstraktions-Ebene.

Test Driven Development (TDD)

Neben Pair Programming ist TDD eine der zentralen Praktiken des **Extreme Programming** (XP). Die Idee dahinter ist recht simpel: bevor Code geschrieben wird, wird ein Test erstellt, der die gewünschte Funktionalität abprüft.

~



Dieses Vorgehen hat klare Vorteile:

- Der geschriebene Code ist besser dokumentiert
- Man hat ein besseres Verständnis über die Verhaltensweise, die Software haben soll (**Requirements**)
- Es ist zu jedem Zeitpunkt klar welche Teile der Software (nicht) funktionieren
- Der Entwicklungsablauf hat eine klare Struktur

Stack und Heap

Der aktuelle Zustand eines jeden Programms wird im Arbeitsspeicher (**RAM**) gespeichert. Die Java Virtual Machine (JVM) verwendet hierfür zwei getrennte Bereiche: den **Stack** und den **Heap**.



Stack

Der Stack ist ein Stapelspeicher, d.h. Informationen können oben auf ihm abgelegt werden und auch nur von oben wieder entfernt werden. Dies folgt dem sog. Last In First Out (LIFO) Prinzip. In der JVM wird folgendes im Stack gespeichert:

- Ketten von Methodenaufrufen
- Werte von lokalen Variablen (d.h. Variablen innerhalb von Methoden oder Blöcken)
- Zwischenergebnisse von Berechnungen
- keinerlei Objekte (sondern nur Referenzen auf Objekte die sich im Heap befinden)

Der Stack ist im Vergleich zum Heap viel kleiner. Wenn sein Speicherplatz ausgeht, wirft die JVM einen **StackOverflowError**. Dies kann ein Zeichen für Programmierfehler im Zusammenhang mit Rekursion sein.

Jeder Thread innerhalb der JVM besitzt seinen eigenen Stack.

Heap

Der Heap ist vergleichbar mit einem Haufen oder einer Wolke, in welcher alle Klassen und Objekte mitsamt ihren Daten gespeichert sind.

Er ist im Vergleich zum Stack viel größer (häufig mehrere Gigabyte groß). Wenn der Heap überfüllt wird, dann wirft die JVM einen **OutOfMemoryError**. Dieses Zeichen für besonders hohen Ressourcenverbrauch kann eine Vergrößerung des Heapspace oder Optimierungsmaßnahmen erforderlich machen.

Alle Threads innerhalb der JVM teilen sich denselben Heap.

Software Qualität

Welche Aspekte zum Begriff **Softwarequalität** gehören, ist in verschiedenen Qualitätsmodellen unterschiedlich definiert. Laut dem Standard **ISO 25010** sind für die Ermittlung von Softwarequalität die folgenden Aspekte/Charakteristika relevant:

- **Funktionale Eignung** Erfüllt die Software ordnungsgemäß und korrekt all ihre Aufgaben?
- **Zuverlässigkeit** Bleibt die Software auch in suboptimalen Szenarien stabil verwendbar?
- **Benutzbarkeit** Ist die Software so gestaltet, dass sie von ihrer Zielgruppe effizient erlernbar/verwendbar ist?
- **Effizienz** Steht der Ressourcen- und Zeitbedarf der Software in gutem Verhältnis zur Funktionalität?
- **Sicherheit** Ist die Software vor unbefugtem Zugriff geschützt?
- **Wartbarkeit** Ist die Software mit vertretbarem Aufwand änderbar/erweiterbar/testbar?
- **Übertragbarkeit** Wie einfach kann die Software auf einer neuen Umgebung installiert/verwendet werden?
- **Kompatibilität** Harmonisiert die Software im Zusammenspiel mit anderer Software?

Diese generell erstrebenswerten Charakteristika werden häufig auch als nichtfunktionale Anforderungen (**Non-Functional Requirements**) bezeichnet – im Gegensatz zu den für jede Anwendung individuellen funktionalen Anforderungen (**Functional Requirements**).

Clean Code

Anders als intuitiv angenommen werden könnte, wird Code deutlich häufiger gelesen, als er geschrieben wird – in „Clean Code“ von Robert C. Martin ist die Rede von einem Verhältnis von über 10:1.

Dabei geschieht der Großteil dieses Lesens durch die selben Personen, die den Code schreiben.

Um bei der Entwicklung nicht mit dem Lesen und Verstehen des eigenen Codes Schwierigkeiten zu bekommen, sollte ein großer Teil des Aufwandes in das Verfassen sauberen Codes fließen.

Eine kleine Auswahl an Tätigkeiten und Prinzipien, die dabei helfen können, werden nachfolgend vorgestellt.

Die Boy Scout Rule

Angelehnt an die Regel der Boy Scouts of America – „Hinterlasse den Zeltplatz sauberer als du ihn vorgefunden hast“ – kann eine einfache Regel für das Arbeiten an Quellcode formuliert werden:

„Hinterlasse Code immer schöner als du ihn vorgefunden hast“

Das muss nicht zwangsläufig umfangreiche Änderungen bedeuten, solange das Resultat den Code sauberer macht.

DRY-Prinzip

DRY steht für Don't Repeat Yourself (wiederhole dich nicht) und ist ein Prinzip, das einen Code ohne Redundanzen propagiert. DRY wird bei Refactorings als eine Möglichkeit verwendet, sinnvolle Anpassungen zu identifizieren. Darunter zählen z.B.:

- Innerhalb eines Blocks wird das selbe Literal mehrfach verwendet
→ Variable/Konstante einführen
- Die selbe Folge von Anweisungen tritt mehrfach auf
→ in eine Methode ausgliedern oder Schleife einführen

S.O.L.I.D.

Unter S.O.L.I.D. werden fünf verschiedene Design-Prinzipien zusammengefasst, die dabei helfen können sauberen Code zu schreiben und Software wartbarer zu machen. Sie sind weniger als Regeln zu verstehen, denen auf jeden Fall gefolgt werden muss, vielmehr können sie eine Leitlinie sein, für die man sich entscheidet. Das Akronym steht für:

- **Single Responsibility**
Jede Klasse sollte genau eine Verantwortlichkeit haben. Das bedeutet aber nicht, dass sie nur eine Funktion erfüllen sollte. Vielmehr bezieht sich diese Aussage darauf, dass jede Klasse nur von einer einzigen Gruppe Menschen verwendet werden sollte.
- **Open-Closed**
Klassen sollten geschlossen für Veränderung und offen für Erweiterung sein. Das bedeutet, sobald Klassen entwickelt sind, sollten sie nicht mehr verändert werden. Statt dessen sollte neue Funktionalität über Erweiterung dieser Klasse implementiert werden (z.B. durch Erstellen einer Unterklasse).
- **Liskov Substitution**
Das Verhalten eines Interfaces (einer Methode), welches einen bestimmten Typen als Parameter entgegen nimmt, sollten Untertypen kein anderes Verhalten hervorrufen als ihre Obertypen.
- **Interface Segregation**
Interfaces (als Sammlungen von Methoden) sollten in ihrer Verwendung isoliert sein. Das bedeutet, dass Klassen keine Abhängigkeiten haben sollten, die sie nicht verfolgen.
- **Dependency Inversion**
Die Implementierung von Software sollte immer von abstrakten Modellen abhängen, statt von konkret implementierten Modellen. Das bedeutet, dass `import` Statements nur abstrakte Klassen oder Interfaces importieren sollten.

KISS-Prinzip

Hinter KISS (Keep It Simple Stupid) steckt die Idee, dass Systeme dann am besten funktionieren, wenn sie so einfach wie möglich gebaut sind. Daher steht KISS für die Erstellung von möglichst einfachen Designs, die keine unnötige Komplexität enthalten.

Software-Entwicklungsmodelle

Für die Entwicklung von nicht-trivialer Software haben sich im Laufe der Zeit (ab den 1960ern) verschiedene **Vorgehensmodelle** etabliert. Diese unterteilen den Prozess der Softwareentwicklung in verschiedene definierte Phasen und/oder definieren verschiedene zugehörige Prinzipien, Rollen, Aktivitäten und Methoden/Techniken/Praktiken.

Es existieren viele verschiedene Vorgehensmodelle, deren Eignung von vielen projektspezifischen Faktoren abhängig ist (z.B. Umfang der Anforderungen, Klarheit der Anforderungen, Größe und Erfahrung des Teams, zeitlicher Rahmen, Budget).

Die derzeit bedeutendsten Ansätze und zugehörigen Entwicklungsmodelle sind:

Iterativer und Inkrementeller Ansatz: z.B. Spiralmodell

Die gesamte Entwicklung wird in kleinere Teilprojekte (Inkmente) aufgeteilt, welche nacheinander umgesetzt werden. Innerhalb jedes Teilprojekts werden die folgenden Phasen solange (mehrmals) durchgeführt, bis ein zufriedenstellendes Ergebnis vorliegt:

Planung → Analyse/Design → Implementierung → Verifikation/Testing

Ein zufriedenstellendes Ergebnis kann eine funktionsfähige Software-Version oder ein funktionsfähiger **Prototyp** sein.

Sequenzieller Ansatz: Wasserfallmodell, V-Modell

Die gesamte Entwicklung wird in die folgenden Prozessphasen aufgeteilt, welche einmalig sequenziell durchlaufen werden:

Anforderungserfassung → Design → Implementierung → Verifikation/Testing → Installation → Wartung

Am Ende des Prozesses steht (hoffentlich) die vollständige und funktionsfähige Software. Das **V-Modell** erweitert das **Wasserfallmodell** um zusätzliche Test-Phasen (*Unit-, Integration-, System-, Acceptance-Test*).

Agiler Ansatz: z.B. Scrum

Die **agile Softwareentwicklung** verfolgt ebenfalls die iterativen und inkrementellen Ansätze, erweitert diese jedoch um eine Reihe von Prinzipien (z.B. „close, daily cooperation between business people and developers“) und moderne Praktiken (z.B. *Backlogs, Test-driven Development, Timeboxing, Pair Programming* und *Continuous Integration*).

Kapitel 4 - Zusammenfassung

- Zur besseren Strukturierung des Programms sollten Klassen und Interfaces in **Packages** geordnet werden.
- Der Zugriff auf Klassen, Interfaces, Variablen, Methoden und Konstruktoren von außen kann über **Visibility-Stufen** (**private**, **package**, **protected**, **public**) eingeschränkt werden.
- Mittels **Imports** wird angegeben, auf welche **externen Klassen** eine Klasse referenziert.
- Um anderen Entwicklenden das Verständnis zu erleichtern, sollte eigener Code immer mittels **JavaDoc** dokumentiert werden.
- Bei der Softwareentwicklung wird zwischen funktionalen und nichtfunktionalen Anforderungen unterschieden. Zu den erstrebenswerten Qualitäts-Charakteristika einer Software gehören: **Funktionale Eignung, Zuverlässigkeit, Benutzbarkeit, Effizienz, Sicherheit, Wartbarkeit, Übertragbarkeit, Kompatibilität.**
- Das Testen von Software stellt sicher, dass möglichst viele Fehler während der Entwicklung gefunden und behoben werden können. Dabei wird zwischen **statischen** und **dynamischen Testverfahren** unterschieden. Als letztere zählen unter anderem **Unit-** und **Akzeptanz-Tests**
- **Test Driven Development** legt den Fokus der Entwicklung auf das Testen als treibende Kraft. Dadurch wird eine hohe Qualität der entstehenden Software ermöglicht.
- **Refactoring** bezeichnet eine Restrukturierung des Quellcodes, ohne dessen Funktionalität zu verändern.
- Mit verschiedenen Techniken und Prinzipien kann Code sauber gehalten werden, um unter anderem Wartbarkeit und Lesbarkeit zu erhöhen. Darunter fallen beispielsweise **DRY, S.O.L.I.D.** und **KISS**
- Die JVM speichert den aktuellen Programzustand in **Heap** und **Stack**.